

Projet 2026 - Blozchain (LINFO1104) : Rapport

Aubecq Quentin (36092400) et Ghislain Tom (36342400)

Mai 2026

1 Structure de la fonction `ExecuteBlockchain`

1.1 Fonctions secondaires

Les fonctions suivantes sont utilisées dans `ExecuteBlockchain` :

- **TransactionHash** : Calcul et renvoie le hash d'une transaction
- **BlocHash** : Calcul et renvoie le hash d'un bloc
- **ComputeTransactionEffort** : Calcul et renvoie l'effort d'une transaction
- **IsTransactionValid** : Affiche un message d'erreur et renvoie `false` si la transaction n'est pas valide, sinon renvoie `true`
- **AddUserIfNeeded** : Ajoute le receiver d'une transaction à l'état de la blockchain si il n'est pas encore présent
- **GetUpdatedState** : Renvoie le nouvel état après avoir effectué une transaction
- **InitState** : Créé l'état à partir de `GenesisState`

1.2 Fonction `ExecuteBlockchainAux`

La fonction `ExecuteBlockchain` contient une sous fonction `ExecuteBlockChainAux`.

Cette fonction va en premier lieu vérifier si l'argument `Num` (numéro du bloc actuel) est plus grande que le nombre de bloc (cf. 1.2.1), dans quel cas la fonction s'arrête, donne une valeur à `FinalState` et renvoie `nil` pour terminer la liste de bloc.

Si ce n'est pas le cas, la fonction récupère le prochain bloc (cf. 1.2.2) et fait un appel récursif (`Bloc|{ExecuteBlockChainAux ...}`) en augmentant `Num` de 1 pour récupérer le bloc suivant.

1.2.1 Fonction `GetMaxBlockNumber`

Cette fonction itère (récursivement) sur la liste des transactions et récupère le plus grand `block_number`.

Pour optimiser la rapidité du programme, on aurait pu appeler une seul fois cette fonction et donner la valeur à `ExecuteBlockchainAux`.

1.2.2 Fonction `GetBlock`

Le fonction `GetBlock` itère (récursivement) sur toutes les transactions. La transaction est ajouté à la liste et effectuée (changement de l'état) uniquement si son `block_number` est bon, si le transaction est valide et si l'effort total du bloc (connu grâce à un accumulateur) ne dépasse pas 300.

2 État final de la blockchain

2.1 État final

User	Balance	Nonce
11	97415	2
14	70294080	8
15	94650918	6
32	5141804	7
37	1239534151	0
49	463119	1
60	36160	3
61	221043	4
66	480590	2
73	58497	4
86	918173	2
87	210757	1
100	579432	3

2.2 Blockchain finale

Bloc	Hash	PreviousHash	Hash des transactions
0	256424	/	22472, 38896, 81865, 61514, 51677
1	153662	256424	363117, 465569, 68551
2	146614	153662	751304, 50438, 62480, 128728
3	647331	146614	291119, 170096, 24257, 15242
4	102336	647331	247790, 139871, 67340
5	272434	102336	92750, 47822, 23514, 6007
6	551027	272434	24505, 181165, 72917
7	297173	551027	479975, 16061, 78363, 133002, 38738
8	552873	297173	87525, 106662, 61505
9	485117	552873	281209, 582818, 68208
10	552350	485117	32842, 11083, 23298
11	732472	552350	16992, 42982, 120137

2.3 Phrase secrète

La phrase secrète est :

prof peter van royiarty is behind oz

3 Extension : Denylist

3.1 Implémentation

Pour l'extension Denylist, nous avons ajouté 2 champs au record `user()` : `block_sends`, qui est un compteur du nombre de transactions effectuées en tant que `sender`, et `denied`, qui est un booléen notant si l'utilisateur fait partie de la Denylist (qui n'a donc pas de représentation explicite dans le code, mais est uniquement représenté par le champs `denied`) Pour ce changement il a donc fallu modifier les fonctions `GetUpdatedState` (ainsi que sa fonction auxiliaire `GetUpdatedStateAux`) et la fonction `InitState` (à nouveau, ainsi que sa fonction auxiliaire `InitStateAux`). Une branche a été ajoutée en tête de `IsTransactionValid` pour rendre les transactions dont le `sender` est dans la Denylist invalides. Enfin, deux nouvelles fonctions nous ont été nécessaires : `GetUpdatedBlockSends`, une fonction prenant un état ainsi qu'une transaction, et qui renvoie un nouvel état, dans lequel la transaction a été comptée dans les `block_sends` des utilisateurs (elle contient une auxiliaire, `GetUpdatedBlockSends`). Ainsi que `GetResetBlockSends`, une fonction prenant en seul argument un état, et qui remet l'entièreté des `block_sends` à 0, tout en mettant `denied` à `true` pour les utilisateurs ayant atteint la limite de 3 transactions ou plus (ayant aussi une auxiliaire, `GetResetBlockSendsAux`). Pour intégrer ces deux fonctions, `GetBlock` a subi deux changements, d'abord un appel à `GetUpdatedBlockSends` à chaque traitement de transaction (valide ou non), ainsi qu'un appel à `GetResetBlockSends` à chaque changement de bloc.

3.2 Etat final

User	Balance	Nonce
11	97415	2
14	70226791	8
15	94043702	6
32	6107292	3
37	1239534151	0
49	463119	1
60	36160	3
61	221043	4
66	457435	2
73	81652	3
86	918173	2
87	210757	1
100	288449	3

3.3 Blockchain finale

Bloc	Hash	PreviousHash	Hash des transactions
0	256424	/	22472, 38896, 81865, 61514, 51677
1	153662	256424	363117, 465569, 68551
2	146614	153662	751304, 50438, 62480, 128728
3	356212	146614	170096, 24257, 15242
4	743877	356212	247790, 139871
5	913975	743877	92750, 47822, 23514, 6007
6	168063	913975	181165, 72917
7	914209	168063	479975, 16061, 78363, 133002, 38738
8	169909	914209	87525, 106662, 61505
9	519335	169909	281209, 68208
10	563270	519335	32842, 11083
11	743392	563270	16992, 42982, 120137

3.4 Phrase secrète

La phrase secrète obtenue avec cette extension est :

prof petezpc h g qh gp ejzjwx xi

4 Extension : L'effort n'est pas gratuit

Cette extension était relativement simple à implémenter, il a fallu modifier uniquement deux endroits dans le programme :

- La fonction *IsTransactionValid* à été modifiée pour ajouter le coût de l'effort lors de la vérification de la balance du sender
- La fonction *GetUpdatedState* à été modifiée pour enlever le coût de l'effort en plus de la valeur de la transaction

La blockchain en elle même ainsi que la phrase secrète n'ont pas changé, en effet aucune transaction n'a été rendu invalide par cette extension.

Les balances finales des utilisateurs ont changé puisque les transactions leur enlève plus d'argent, voici le nouvel état final :

User	Balance	Nonce
11	97353	2
14	70293640	8
15	94650668	6
32	5141523	7
37	1239534151	0
49	463088	1
60	36035	3
61	220919	4
66	480496	2
73	58341	4
86	918127	2
87	210726	1
100	579275	3